Python for computational Problem Solving · Unit_04 · UNIT 4: Testing ,Debugging and Object Oriented Programming

DRAFT — UE26CS151A · Unit_04 · UNIT 4: Testing ,Debugging and Object Oriented Programming · Exceptions

Standalone video lesson · authored study notes with source citations

Hugging Face PESU_Materials source tree, official units.json catalogue, transcript sidecars, slide text/PDF/PPT sources and the selected First_Year_8enterprise archive; see the provenance datasets in the release ZIP. Generated 18 October 2026.

· How to use this pack

Read the topic explanation, worked examples, and recap. Evidence citations point to source transcript intervals and slide pages; full slide images remain in the source visual index. Topics include the applicable parts: overview, key terms, formulas, figures, flowcharts, important points, worked examples, past-paper questions and a recap. IDs are shared with the separate documents: E = formula, G = graph, F = flowchart, W = worked example. Each ID is numbered by topic, so E4.2 is the second formula of topic 4.

Source text	Verbatim or OCR-derived text, kept separate from editorial notes.
Candidate	Automated topic/source association requiring manual confirmation.
Low ASR confidence	Transcript segment requires audio verification.
DRAFT	Human review remains incomplete.

Contents

1. 1 · Exceptions
2. Appendix A · Symbol glossary
3. Appendix B · Sources, coverage and gaps
4. Appendix C · Revision and retrieval practice

M1

Exceptions

1 Exceptions

Study focus: Exceptions. This topic explains the central idea, its conditions, and how the lecture applies it in context. Review the selected source statements and follow their citations for the full lesson segment. EV0000062 · EV0000063

SHORT-NOTE RECAP

Core explanation · 00:00–04:59

We will see one more small example.

Key details

- Unless you pay attention.
- Can see here I have created a object called.
- Taking the fruit name and the count of the fruits.
- Or a static attribute?
- Is this count?
- I have two count.
- So this count just belongs to that one particular object that we have created because it is self.count.
- But when we look here.
- I am saying fruits.com.
- We know that fruits is nothing but my class name.
- root and I have template of sum of total number of roots.
- It will reflect basically when I try to access this roots.com, obviously I'll get the simulative so apples i'll get apple count as three and food count as three where i make another object piece you can see that it is name of the food count as four now food star count should be three plus four seven is it clear any doubts there.
- When I say peach dot count, I'll get it as four.
- When I say fruits.count, we will get it as seven.
- Four plus three seven because here you can see I made a cumulative stuff.
- If at all I was only thinking of a local variable that is non-static variables, I cannot do cumulative sum like this because it does not carry the same value across all the objects.

- So here I can do it because this is a static variable.
- The apples is an object.
- I'm trying to access the class name of my object apples.
- This is another syntax to access your class variable to the static variable, and you can see a type of A.
- What do I get say if I take a variable A equal to 10 and I check for type of A?
- What will be type of A?
- It would be.
- Class is a user defined type, right?
- It will be.
- It's fruits, right?
- In our case, it's fruits.
- Apples classes foods of similarly type software.com any doubts.

Example or application from the lecture

- So this is a typical example where you can make out difference between a static variable and a normal variable.
- Is it clear you need to be familiar with all possible syntaxes because when you will be given questions to find out or error, if you don't know these syntaxes, then you'll end up not writing it properly.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000062 · EV0000063 · EV0000064 · +1 source units in pack.json

Core explanation · 05:00–09:59

Comment that operator overloading.

Key details

- Apples.count three, apple peach.com four, fruits.count seven, and then apples.class.count seven, and begin type of peaks.count seven.
- Any doubts or any clarification?
- Take down a question quickly.
- create a class create a class marks create a class calculate marks.
- In which in which a student's in which a student's individual subject marks.
- For ISA 1, ISA 2 and ESA.
- Each individual subjects, ISA one, ISA2 and ESA Marks is initialized.
- [ASR low confidence; check audio].
- [ASR low confidence; check audio].
- ISA 1, ISA 2, and ISA marks for.
- Accept ISA 1, ISA 2, and ISA marks for.
- Chemistry Python Max.
- Civil and electronics and electronics find the total marks score find the total marks score and.
- Also find and also find percentage and percentage.

- Did all of you get an idea why will you be doing this?
- You will create a class called as calculate marks, right?
- And in your init function you will be taking ISA one, ISA2, and ESA Mark.
- How many objects?
- How many objects will you get to get?
- How many subjects have I taught?
- I'll not write the entire code that you people write just to tell how you should be using a class variable only that I'll tell.
- This is your class.
- I will just say should be taking [ASR low confidence; check audio].
- You make respective assignments, right?
- Or five subjects.
- Yeah, she looks like this.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000066 · EV0000067 · EV0000068 · +1 source units in pack.json

Core explanation · 10:00–14:59

Every time say I'll get ISA one, Isa 2, and ESA for math, similarly to Python, etc.

Key details

- And then in my can I write what should I write to access total marks here?
- [ASR low confidence; check audio].
- All these right? self.isa1 plus self.isa2, etc.
- [ASR low confidence; check audio].
- Yes, under the internet country.
- No, but still you need to take self.
- Because this is with respect to math object, right?
- No, you need to be whatever I write here though that if at all this I discussed when I did the first program if at all I you might take like this dot.
- A1 is equal to IC1, then it is self dot A1.
- It is not ISA1.
- These are your variables.
- Any clarifications to complete this?
- We'll go to static methods now.
- Basically executing for which particular object.
- That's why my I pass a self here, right?
- I need to make out that accordingly it will execute so we know that this is with respect to objects, so it will be for respect to objects only.
- But when you're looking at these static methods, these are the common methods across any objects.
- Like how we are discussed every method inside a class will have one parameter by default that is cell that will take name of the object, right?
- With respect to both static method and static variables, I have done this.

- Why because I had to tell you all dictionary.
- If at all I say self-taught dictionary, what happens is that all its variables become keys and whatever value I've assigned will become value.
- So should this dictionary look like it should be name because name is my variable name colon whatever I have passed ram here, right?
- Why because with respect to each object every time I'm.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000070 · EV0000071 · EV0000072 · +1 source units in pack.json

Core explanation · 15:00–19:59

equal to one so count will be equal to one now student dot count is a class variable or the static variable so it will count number of students okay as given application like accept details of a student.

Key details

- One after the other and also get me the total count of the students who have registered.
- If this is the case, then this give me total number of students.
- So now you can see I have called the static method display count.
- Okay, I have called the static method display count.
- When I go here, it basically accesses whatever is necessary for the student document.
- Okay, and you can see here this one that is at the rate of static method, it's only optional, but this is optional when you are actually calling your method like this but if you want to call your method with respect to object then you need to be giving this syntax that is the decorative syntax any doubts any doubt is it clear.
- And this is with respect to.
- Please do not write your assignments in the class.
- Any other subject assignments, if at all you are writing, please do not write in the class.
- If at all if somebody is writing, please do not write.
- Any clarifications, any doubts?
- My everyone are looking very dull.
- You had a lot of work.
- If you don't write it's okay, anyway I'll be sending this document.
- Just see that you understand.
- This is a simple program where I've used the static method and I'm finding if a person has a second name or not.
- All of this logic, right?
- Basically, when I get a string, I can split based on spaces.
- string complete string, right?
- Scott will be one string and robin will be second string.
- Just remember this.
- Remember this logic.
- And the last topic before we end, take down composition.
- You have seen inheritance, right?

Example or application from the lecture

- I just gave as an extra example, that's all.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000074 · EV0000075 · EV0000076 · +1 source units in pack.json

Core explanation · 20:00–24:59

The parameter A that means to say that B is a derived class and A is a space class.

Key details

- Here it is something like a class as another class within itself.
- I have two independent classes.
- A and then say I have class B.
- If class B creates an object of class A, that means that object can actually access all the characteristics of my class A.
- You can see here I have a class called as component.
- And I have a method called as M1.
- All simple methods.
- I've just written the phrases.
- Nothing logical in this.
- What is that it makes it as an object composition specifically.
- I'm saying self dot object one is equal to component.
- That means to say that I am creating an object of one class inside another class.
- It composes the characteristics of some other class.
- Because I have created an object of that class inside.
- Okay, this is possible because I had created an object obtain of type component.
- Any doubts, people?
- It says component class object created that happier self.object one.
- And then I basically make HBJ2.
- So m2 method is called, and then inside my m2, I have called m1 method, so m1 method is called actually m1 is a method that belongs to my other class.
- You want to write to understand?
- Or if it's okay the next topic.
- I'll just say this, you can analyze once.
- Difference between inheritance and composition is that in inheritance you will not create any object of the other class, but you will need that inherited as a parameter.
- You have not inherited, but you have created object of another class.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000078 · EV0000079 · EV0000080 · +1 source units in pack.json

Core explanation · 25:00–29:59

S students are present now.

Key details

- It's Vincent, Aditya, Krishna, Anshal, Abai, R, Abhishek, yes.
- Abhishek is Kupri.
- You were present.
- Morning you were absent.
- Please remove your mask when you present morning.
- You can just recollect.
- Objects are basically those variables which will allow you to access the characteristic features of the class, or you can also say object is an instance of a class, and then functions inside a class by default it will have variable parameter which is called a self unless and until it is a static method okay you studied about constructors you studied about parametrized and non-parameterized constructors.
- And then you studied about these default functions which are automatically accessed when you pass this can be access when you pass a string to another function, right?
- And then you studied about access modifiers, public and private, right?
- By default, what are the type of methods and variables inside a class?
- It is private All of you are so colour.
- It will be what public or private?
- By default it will be.
- Otherwise if it needs to be private, then you need to prefix the underscore, underscore.
- Then you studied about inheritance.
- You studied about multiple inheritance.
- And then you studied out to call in it function, huh?
- Base class, from base class, I mean from derived class, you also call base classes in its function, and you also studied about super method, which basically allows you to access methods and in it method from your base class.
- You studied about static variables, static methods, operator overloading, and then you studied about composition and you also studied about.
- Function overloading, right?
- Or method overloading.
- Remember all these concepts.
- Any clarification among all these topics?
- Is it clear?
- Is it clear to all of you?
- Hope you have been doing your own work problem that I'm giving it take only 10 to 15 minutes.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000082 · EV0000083 · EV0000084 · +1 source units in pack.json

Core explanation · 30:00–34:59

Take down the next topic exception handling.

Key details

- As the name says when we say exception, always it'll be normal except for certain reasons it might not work.
- Something like in your timetable.
- Whichever subjects are there, those subjects will be taught on those respective days unless and until the faculty is absent.
- What are more errors we have?
- We have syntax errors, we have semantic errors.
- We don't have any problem with syntax errors, because your interpreter is capable of handling it.
- When it comes to your semantic error, we will have to deal with it.
- We saw by test and we saw it basically tells us where exactly we have gone wrong.
- One more way is your exceptions.
- It basically tells us that how we need to handle those semantic errors.
- Something like when I try to divide a number by zero.
- And in that exception, all these inbuilt exceptions are being defined.
- Something like divide by zero, index out of form.
- And I try to enter something about two hundred that cannot be anticipated by your interpreter, right?
- Because that is application oriented.
- My application maximum marks is two hundred.
- it need not be same always so such application dependent semantic errors should be handled by the programmer itself okay by writing a user defined exception is that clear so.
- Basically, when I when we talk about exception handling, we will think about the errors.
- This is shown by my interpreter itself.
- If I take AB and I say C equal to AB to give me a error, right?
- I told about written exceptions and user defined exceptions.
- It's not necessary that I should be using the same message.
- I can also override on this like how we did overriding of underscore add like that you can also overwrite these exceptions, but in bed I have certain phrase about these exceptions.
- Zero division error.
- I'm trying to divide by zero.
- Then we have IO error.
- Then I told about index error.
- I don't have a variable called as byte.
- But still I try to print my that means that it's a name error because I don't have a variable of that.
- And we have a type error.
- I'm trying to add a string and a number, right?
- I cannot do that because there's a type error.
- I cannot add a string and a picture.
- And we have something called as value error.

Example or application from the lecture

- And say for example I have a list, I have five elements in the list, I'm trying to access the sixth element in the list.
- All these exceptions have been given.
- But say for example, I'm basically taking your map and maximum marks is two hundred.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000086 · EV0000087 · EV0000088 · +1 source units in pack.json

Core explanation · 35:00–39:59

So we'll see if you have the example executed here.

Key details

- I have x equal to 10, it's an interactive mode, and I try to print y.
- Okay, so you can see here I have a name error because Y is not defined.
- Here I'm taking a number and this is also a number but it's in form of a string, right?
- Because type mismatch.
- Here I have index.
- There's no where number call is 40 in my list, but I'm trying to find an index where 40 resides.
- I'm trying to access some value which is not there, so it's a value error.
- And if I'm trying to say in this case itself, I try to access say L of four.
- I have zero one two index.
- If I try to access L of 4 index, fourth index is not there, so it will be index out of bounds exception.
- These are the common things that you need to remember.
- Then I have a list try to access this.
- I've got axe of the form.
- And then A plus 10, it's a name error because I don't have a variable called A and I'm trying to increment it by 10.
- Name error, tenure, two and six.
- I cannot multiply two strings, so it's the type error.
- Here I cannot convert this floor which is in form of a string to an integer.
- I cannot type cast this, right?
- These are all the exceptions.
- I'm just going fast because it's simple and nobody's asking me anything.
- Simple, what we will do is that I have a program.
- You will we can use we will use a keyword called as except.
- Which basically this block of code which is under accept will get executed only if an exception is raised in the right block.
- I try to say it's A divided by B, I accept both values of A and B.
- And if it is say A is 100 and B is 20, so there's no exception, right?
- But when I give value of A as 10 and value of B as zero, then there will be an exception at that point of time.
- Set up instruction which is under it, okay?

- [ASR low confidence; check audio].
- We will just see what happens if I just executed it as it is.
- This is what we get.

Example or application from the lecture

- A simple example, file print x, an exception occurred as it is, not the fight.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000090 · EV0000091 · EV0000092 · +1 source units in pack.json

Core explanation · 40:00–44:59

When I did not use except, this is your error is shown by your interpreter.

Key details

- Obviously, if you are writing a application for your client, he would not like to see this.
- Because he doesn't understand what this is.
- Unless Until he is a programmer, he will not understand what this usually clients say it will be non-programmer, right?
- This is not handle, this is.
- No, this is here I had handled it like this, a tri block and a accept block.
- I can also handle it by giving our.
- Inbuilt exception name.
- Say for divide by zero, I have inbuilt exception.
- This is how we can add it by using [ASR low confidence; check audio].
- Inbuilt exception, this is how we end.
- Here again we have handled the name error.
- I'm dividing it by zero.
- Okay, so basically it is giving me the error because it's not a name error, right?
- Actually y equal to one divided by zero was a zero division error.
- But while writing my except block, I wrote I am.
- Use a wrong inbuilt exception.
- Say my trial block has some 10 instructions.
- In that I can accept that could be a divide by zero, that could be an index of, it could be something else as well.
- I can have any number of except blocks.
- For a single tribal, you can have any number of except blocks.
- In this case, when an exception was raised, so divide by zero error was raised.
- And then you can see out of.
- Try accept block.
- It doesn't mean that there's an exception, it will not execute the rest of the code, it will still execute the rest of the code.
- Can you doubt?
- So it will first look for the first match.

- Say something like there are set up except block, so it will check the exception which has got raised.
- Does it belong to this category?
- It doesn't, then it goes to next category.
- And then it goes to next category.
- Like you have two similar kinds of exceptions which is occurred.
- In that case, it will take the first exception block which is there.
- It will not look for the best exception block.

Example or application from the lecture

- How does it basically consider which except block to be executed when there is some exception?

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000094 · EV0000095 · EV0000096 · +1 source units in pack.json

Core explanation · 45:00–49:59

It will look for the first matching except not.

Key details

- Here A is 5 and B is 6.
- With except you can have a else block.
- One pride block can have any number of except block.
- Basically, this is when no error occurs.
- Earlier, what used to happen?
- If the error occurs, it goes to next step block.
- But if error does not occur, you want set of operations to be done, then you can put it under else block.
- Is that clear?
- When do we use the else block?
- If at all you have two things to decide if a error, if an exception occurs, what to do.
- And if exception does not occur, what to do, then what to do if an exception does not occur, you can put it under else block.
- This is basically raising our own exceptions.
- Till now you have seen all built-in exceptions.
- I'm trying to enter marks with greater than 200.
- That is not input exception, so I have raise my own exception.
- Please take syntax of this.
- Take syntax of your user defined exception.
- Abhishek, what are you doing?
- I don't see.
- I hope you have taken the normal inbuilt exception handling, right?
- Try and then except.
- You can try it any number of except.

- [ASR low confidence; check audio].
- And set off except with else, yeah.
- Can you just remove your mask?
- [ASR low confidence; check audio].
- At what time?

Example or application from the lecture

- Say, for example, the example that I gave, Marks is maximum marks is 200.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000098 · EV0000099 · EV0000100 · +1 source units in pack.json

A Symbol glossary

Symbol	Name	Meaning	Appears in

B Sources, coverage and gaps

Source	Path	Use
Exceptions transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/015_Exceptions__524460b511d1f7c5.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Exceptions transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/015_Exceptions__524460b511d1f7c5.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Exceptions transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/015_Exceptions__524460b511d1f7c5.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9999; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
Exceptions transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/015_Exceptions__524460b511d1f7c5.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Historical question-source excerpts are shown only when a page has a strong text-based unit match. They are source references, not new solutions or exam predictions.

Known gaps, conflicts and corrections

1. No historical archive question page met the unit-match candidate threshold; all subject-level and ambiguous pages remain in datasets/question-archive.json.
2. 1 video-to-slide associations were not promoted to a match. Candidate rankings are listed in the transcript corpus and video-slide-matches.csv; the slide decks appear separately.
3. 10 transcript segments have low ASR confidence; they are flagged with timestamps in the transcript blocks and indexed in the corpus.
4. This is a visibly labelled source-preservation draft. Human page-level semantic and visual review is not complete; no unsupported subject summary or formula correction is claimed.
5. The 19.26 GiB MP4 files remain hosted on Hugging Face; the full video path/hash and transcript coverage are indexed. The full audio/video was not watched end-to-end.
6. Formula candidates are machine-extracted text with source locators; use the adjacent original slide page and the formula index before treating a candidate as an authoritative equation.
7. The study PDF contains edited, topic-organized lecture notes and textual slide-page takeaways. Full-page slide images remain in the source/visual index datasets, not embedded as the lesson. Human content, ASR, formula, and page-level visual review remains pending.

Coverage

Evidence is summarized by source to keep the reading copy legible. Full source-unit IDs, locators and decisions remain in pack.json.

Source	Unit counts	Example decisions
SRC00018	10 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0042. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0043 to s0063. Sidecar variants compare...
SRC00019	10 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0042. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0043 to s0063. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00020	10 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0042. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0043 to s0063. Sidecar variants compare...
SRC00021	10 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0042. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0043 to s0063. Sidecar variants compare...

C Revision and retrieval practice